
RECURSO · FACUNDO CORENGIA · BASDONAX

n8n o Código:Cuál Usar para Cada Proyecto

La verdad después de más de 30 proyectos: por qué en producción el resultado es casi el mismo, y cómo decidir con qué armar tu agente según lo que ya sabés.

GUÍA PRÁCTICA · PARA APLICAR, NO PARA MIRAR

EMPEZÁ POR ACÁ

n8n vs código: la pelea que casi no existe

Un montón de gente te dice que hace agentes de IA con Claude Code en 30 minutos y facilito. Otros te dicen que no, que lo mejor es hacerlo con n8n. Y ninguno te muestra lo que pasa después, en producción, con un cliente real adentro.

Yo desarrollo desde 2023. Estuve casi dos años haciendo agentes a mano, con código puro y LangChain. Después me pasé a n8n. Y cuando salió Claude Code me volví al código, pero ahora escribiéndole a una IA en vez de tipear todo yo. Al final terminé haciendo lo mismo que hacía antes, solo que de otra forma.

Entre desarrollos y consultoría llevo más de 30 proyectos de IA, muchos son agentes en producción. Así que esto no te lo cuento de un tutorial: lo implementé en clientes reales, de las dos formas.

Este recurso es para el que ya tiene un negocio y está metiendo IA, o el que la implementa en otros negocios y sabe algo de código. La pregunta no es cuál es mejor 'en teoría'. Es cuál te conviene a VOS, según lo que ya sabés hacer.

Spoiler honesto: en producción, el resultado de un agente hecho con n8n o hecho con código puro es mínimamente el mismo. Toda la diferencia está en la barrera de entrada y en qué haces con los extras.

LO PRIMERO

Un agente no es 'la IA'. Es un flujo con IA adentro

El de 5 minutos que te muestran, ese donde ponen un nodo, le meten la IA y listo, no es un agente. No lo hablás ni por WhatsApp. Un agente de verdad es un flujo con una IA adentro. Eso mismo lo dijo el dueño de Vercel en un podcast: es un workflow, no 'una IA'.

¿Cómo te das cuenta si lo que te muestran es humo o es real? Fijate el primer nodo del flujo. En uno de verdad, llega el mensaje de WhatsApp, pasa por Meta, entra a un CRM de conversaciones (Chatwoot), se procesa en n8n y recién ahí sale la respuesta. Hay un montón de cosas que contemplar: qué pasa cuando llegan varios mensajes juntos, cómo hacer que quede humano, qué símbolos quedan mal.

Un proyecto real que tengo en producción tiene tres agentes con un triage que decide a cuál va cada conversación (uno para el que recién arranca, otro para el que ya vende, otro para el cliente recurrente), manda un mail cuando entra un humano, y hasta agarra los mensajes que le escribís mientras está respondiendo y los deriva a otro agente para contestarlos cuando termina. Eso no lo armás en 5 minutos.

El agente 'facilito' que ves por todos lados no tiene memoria, no tiene CRM, no maneja mensajes en paralelo. Un agente que aguanta producción, sí. Esa es la diferencia que casi nadie te muestra.

Por qué n8n te la banca (y para quién es)

Estos son los puntos fuertes reales de n8n que noté después de tenerlo años en producción con clientes. No es la opción 'de juguete': es la que uso en la mayoría de mis proyectos.

Es fácil de entender

Trabajás con nodos, lo ves. Si sos vos o un cliente el que lo va a mantener, no tenés que saber programar para leerlo y modificarlo.

Le dejás flujos prearmados

Al cliente le podés dejar el flujo hecho y decirle: 'solo armate el prompt y las herramientas, no te preocupes por el resto'. Se centra en la conexión de Meta, el prompt y las herramientas. Nada más.

Aguanta producción de verdad

Tengo clientes recibiendo 1.000 y 2.000 mensajes por día y anda perfecto. Nunca me pasó que 'se rompa todo'. Si lo haces bien, no vas a tener problema.

Con Claude Code lo armás mucho más rápido

El flujo complejo que te describí tardé desde fines de 2024 hasta hoy en conseguirlo. Y lo terminé de lograr también gracias a Claude Code, que te ayuda a armar los flujos y un montón de cosas.

El mito de que 'se rompe todo'

Alguien que nunca implementó en una empresa real te va a decir que n8n anda mal, que es la peor idea, que se te cae el servidor. En mis proyectos, con agentes en producción recibiendo miles de mensajes, eso no pasa. Fijate a quién le vas a creer: al que lo hizo o al que solo lo escuchó.

EL LADO CÓDIGO PURO

Qué te da LangChain (los 'chiches' que n8n no)

Cuando te metés en código puro con LangChain, se te abre un montón de opciones. La verdad que está muy bueno. Estos son los extras reales que te da y que en n8n no tenés tan a mano.

Todas las memorias

Memoria a corto plazo, memoria a largo plazo, las que quieras. Le podés meter cosas para que recuerde los datos del cliente entre conversaciones.

Evaluations y tracking

Podés evaluar y chequear cómo va trabajando el modelo, ver qué herramienta ejecuta, qué está bien, qué está mal, y tener todo trackeado de qué está pasando en cada ejecución.

Anda más rápido

Te anda mejor y más rápido que n8n, y a largo plazo la IA puede ser mejor porque la tenés evaluada y medida de verdad.

La documentación la entiende la IA

LangChain está preparado no solo para que lo leas vos, sino para que la IA lo entienda: le pasás su llms.txt a Claude Code y ya sabe todo lo que tiene que hacer para armar el agente.

La gran barrera de entrada

Todo eso suena hermoso, pero código puro tiene una barrera de entrada enorme. Si te falta la base, vas a tardar mucho más en entender todo esto, y al final obtenés un resultado igual al de n8n. Todos los chiches no los vas a aprovechar, porque el tiempo se te va en entender, y cuando entendiste ya se te terminó el mes del proyecto.

EL FILTRO REAL

Estas cosas tenés que saber SÍ o SÍ para ir a código puro

Esta es la lista concreta que menciono en el video. Si te falta la mayoría, no vayas a código todavía: no es que 'aprendas por el camino', es que el proyecto se te muere antes de arrancar. Si ya las tenés por haberlas usado antes de que existiera la IA, metete de cabeza.

- ✓ **Git y GitHub:** qué es un pull request (PR), la rama de producción y la de testeo, y haber trabajado en equipo con esto antes.
- ✓ **Un entorno de pruebas:** lo más parecido posible a producción, para no romper lo que ya anda.
- ✓ **Editores de código:** moverte cómodo en un editor (yo uso VS Code) y entender lo que hace la IA para poder decirle 'esto está mal, no tiene sentido'.
- ✓ **Modelos de IA y prompts:** saber de modelos y escribir prompts bien hechos (esto también aplica a n8n, ojo).
- ✓ **Docker, VPS y Python:** saber qué es Docker, qué es un VPS y haber usado Python, idealmente desde antes de la IA.



APIs y seguridad: saber de APIs, limitarlas con seguridad, hostear el código en un servidor y protegerlo para que no te lo revienten.

EL VEREDICTO

Cuál elegís vos (la regla simple)

Si NO sabés de código, o si el que va a mantenerlo no sabe: n8n. Vas a obtener un buen resultado, no se te cae el servidor y podés salir a resolverle el problema al cliente sin morir en el intento.

Si YA sabés las bases (Docker, VPS, Python, Git, editores, trabajaste en equipo): metete de cabeza en código puro con LangChain y Claude Code. Vas a armar un buen plan, la IA se mete, trabaja, y tenés rápido el entorno de pruebas y todo lo que necesitás.

Yo el próximo proyecto de agentes lo voy a hacer en código puro con LangChain para producción, sobre la misma base del proyecto que ya tengo andando. No porque el resultado vaya a ser mejor de entrada (va a ser mínimamente el mismo que n8n), sino por lo que puedo hacer con los extras: más control de los datos y una IA que mejora a largo plazo porque la tengo evaluada.

La pregunta no es 'cuál es mejor'. Es: cuánto sabés vos. Si sabés mucho, es lo mismo o mejor código. Si no, es n8n. La herramienta la elige tu nivel real, no el hype.

PARA APLICAR

Dos jugadas concretas que menciono en el video

Dos formas reales de trabajar que salen del video, para que no te quede en teoría. Copiá la lógica, no el ejemplo.

BLOQUEAR HERRAMIENTAS Y ORDENARLAS

En vez de tocar el prompt cada vez, hacé que cuando una herramienta ya se ejecutó quede **bloqueada**, y que el sistema le diga a la IA: 'esta ya se ejecutó, la siguiente es esta'. Así las herramientas van en el orden correcto sin que la IA se mande cualquiera. Muchas veces la solución no es cambiar el prompt: es controlar el flujo.

DELIRIOS AUTO-SOLUCIONADOS

Juntá los errores o 'delirios' que te reporta el cliente en un **Google Sheets**. Con Claude Code (o algo tipo agente que corra solo) los vas detectando y solucionando en automático: se arregla, se marca como hecho, y seguís con el siguiente. En código puro, sumale las evaluations de LangChain para que analicen el último día de ejecuciones y te avisen dónde hay delirios.

El flujo completo de cada uno lo muestro andando en pantalla en el video.

Elegí tu herramienta y salí a hacerlo

Ya sabés la verdad: en producción el resultado es casi el mismo, y lo que decide es tu nivel. Ahora la pregunta es qué haces con eso. En el video te muestro los dos lados andando en pantalla, con el proyecto real de un cliente adentro.

Lo cambia todo?



vs



Claude Code vs n8n: Toda la Verdad

El video completo, con el flujo real de un cliente en producción y las dos formas comparadas en pantalla.

► Ver el video

Si querés que veamos tu caso y demos el primer paso juntos, escribíme directo. Te respondo yo.

Hablame por WhatsApp →

facundocorengia.com

FACUNDO CORENGIA · BASDONAX